Infraestrutura de Software

Implementação 4 — Sincronização do Metrô

Relatório

Beatriz Loyola Gomes de Vasconcelos
blgv@cesar.school

4 de novembro de 2026

1 Objetivo

Este relatório documenta a implementação das funções de sincronização de uma estação da METRO-REC, em que cada passageiro e cada carro são threads POSIX independentes: a `struct station` e as funções `station_init`, `station_wait_for_car`, `station_load_car` e `station_on_board` garantem que um carro que chega com `count` assentos livres só retorne de `station_load_car` quando todos os passageiros que embarcaram estiverem sentados e o carro estiver cheio ou não houver mais ninguém esperando.

O executável `sync_car`, gerado pelo `Makefile` com o `sync_car-runner.c` do professor, imprime `Looks good!` quando o comportamento é o especificado.

2 Checklist de Requisitos

Requisito do enunciado	**Status**	**Como testar / evidência**
Implementado em C para Linux, usando só `pthread_mutex_init/lock/unlock` e `pthread_cond_init/signal/wait/broadcast`	OK	`grep -n "pthread_" sync_car.c` lista só essas sete; compilado e executado em Ubuntu 24.04 (README)
Sem semáforos nem outras primitivas	OK	`grep -n "sem_" sync_car.c` sem nenhuma ocorrência
No máximo um lock por `struct station`	OK	`sync_car.h`: um único campo `pthread_mutex_t mutex`
`station_wait_for_car` só retorna com carro na estação e assento livre	OK	Runner (`Looks good!`, itens 3 e 4); caso passageiros(10) > assentos(3) (item 16)
`station_load_car` só retorna com todos sentados e carro cheio ou sem ninguém esperando	OK	Runner confere `car departed ... (expected N)` a cada carro (itens 3 e 4, Figura 2); casos limite (item 16)
Passageiros embarcam simultaneamente (`station_wait_for_car` retorna para vários antes de qualquer `station_on_board`)	OK	Runner conta os retornos em `threads_completed` antes de chamar `station_on_board` (`sync_car-runner.c:127-131`); item 4
Sem *busy-wait*	OK	Todas as threads em `S` e 0,0 % de CPU, com passageiros e com carro bloqueados (itens 19 e 20)
Compilação por `Makefile`, gerando `sync_car`	OK	`make clean && make` (item 2); só warning do runner

Requisito do enunciado	Status	Como testar / evidência
`Looks good!` do runner de forma estável	OK	100 de 100 execuções (item 4)
Sem data race em `sync_car.c`	OK	`-fsanitize=thread` e `helgrind`: avisos só em `sync_car-runner.c` (itens 7, 8, 11 e 12; limites na Seção 6)
Caso limite e caso inválido	OK	8 casos, incluindo `count=-1` (itens 16 e 17, Figura 3)

3 Como Reproduzir

Compilar:

```
make clean
make
```

Executar:

```
./sync_car
```

Gera as linhas `car entering station .../car departed station ...` de cada carro e termina em `Looks good!`.

Validar (100 execuções):

```
ok=0; fail=0
for i in $(seq 1 100); do
    if ./sync_car > /dev/null 2>&1; then
        ok=$((ok+1))
    else
        fail=$((fail+1)); echo "execucao $i FALHOU"
    fi
done
echo "ok=$ok fail=$fail"
```

A saída esperada é `ok=100 fail=0`.

ThreadSanitizer (com ASLR desligado só no processo, ver item 6):

```
gcc -fsanitize=thread -g -Wall -Wextra -pthread \
    -o sync_car_tsan sync_car.c sync_car-runner.c
setarch "$(uname -m)" -R ./sync_car_tsan
```

Helgrind (cópia do runner com `alarm` de 120 s; o original dá *timeout*, itens 9 a 14):

```
mkdir -p /tmp/hg
cp sync_car.c sync_car.h sync_car-runner.c /tmp/hg/
cd /tmp/hg
sed -i 's/_alarm(1,/_alarm(120,/; s/_alarm(2,/_alarm(120,/' \
    sync_car-runner.c
gcc -g -Wall -Wextra -pthread -o sync_car_hg sync_car.c sync_car-runner.c
valgrind --tool=helgrind --fair-sched=yes ./sync_car_hg
```

Os programas de teste próprios (`teste_limite`, `teste_busy`) não fazem parte da entrega; seus resultados estão nos itens 16 a 20 do diário e no `evidencias.log`.

4 Arquitetura (resumida)

Arquivos e responsabilidades:

1. `sync_car.h` → `struct station` e protótipos das quatro funções, com *include guards*.
2. `sync_car.c` → `station_init`, `station_wait_for_car` (passageiro), `station_load_car` (carro) e `station_on_board` (passageiro sentou).
3. `sync_car-runner.c` → teste do professor, sem modificação: cria 100 threads de passageiro e carros com número aleatório de assentos, protegidos por `alarm`.
4. `Makefile` → alvos `all` e `clean`; compila `sync_car.c` e `sync_car-runner.c` com `-Wall -Wextra -pthread` em `sync_car`.

Decisões técnicas:

1. **Reservar o assento dentro de `station_wait_for_car`** (`free_seats--`, `boarding++`), não em `station_on_board` → sem isso, dois passageiros acordados pelo `broadcast` levariam o mesmo último assento antes de qualquer um chamar `station_on_board` → ao custo de um terceiro contador (decisão 2).
2. **Contador `boarding`** → o carro precisa saber não só quantos esperam, mas se ainda há alguém a caminho de um assento já reservado → `station_load_car` só retorna com `boarding == 0`, e `station_on_board` só sinaliza `all_boarded` quando o contador zera, evitando acordar o carro a cada passageiro.
3. **`pthread_cond_broadcast` em `car_arrived` e `while` (nunca `if`) em todo `pthread_cond_wait`** → pode haver mais passageiros do que assentos, e todos precisam reavaliar a condição; quem não achou assento volta a dormir, e o mesmo laço cobre *wakeup* espúrio → ao custo de acordar todos os que esperam a cada carro, mesmo os que não conseguirão sentar.
4. **`car_present = 0` e `free_seats = 0` ao final de `station_load_car`** → passageiro atrasado não embarca em carro que já partiu → quem chega depois do carro cheio dorme até o próximo carro.
5. **Um mutex e duas condvars, uma por tipo de espera** (`car_arrived` para passageiros, `all_boarded` para o carro) → exigência do enunciado (um lock, sem semáforo), e separar as condvars evita acordar o grupo errado.
6. **`count <= 0` ou `waiting == 0` retornam imediatamente** → o runner exige retorno imediato sem passageiros ou sem assentos. Como as funções são `void` e não há como reportar `count` inválido, o negativo é tratado como zero.

5 Estratégias e Diário de Desenvolvimento

5.1 Estratégias da semana

Estrat.	Nome curto	Contexto	Motivo da troca
S1	Mutex e duas condvars, com reserva de assento em `station_wait_for_car`	Um `pthread_mutex_t`, duas condvars (`car_arrived`, `all_boarded`) e três contadores; decisões 1 a 6 da Seção 4	(ponto de partida)

5.2 Diário de Tentativas

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência (`grep -a` em `evidencias.log`)
1	S1	Compilar `sync_car.c` com `gcc -Wall -Wextra -pthread -c` enquanto escrevia `station_wait_for_car`	Falha parcial → corrigida: `implicit declaration of function 'cond_wait'` em `sync_car.c:26`; após a correção, compilação sem saída	Usei o nome do pseudocódigo (`cond_wait`) em vez de `pthread_cond_wait`	03/11 (17:51–18:54)	`implicit declaration`
2	S1	`make clean && make`	OK: único warning é do runner (`sync_car-runner.c:54`, `unused parameter 'foo'`); `sync_car.c` sem warnings	—	03/11 18:41	`unused parameter`
3	S1	`./sync_car`, uma execução	OK: `Looks good!`	—	03/11 18:46	`Looks good`
4	S1	Loop de 100 execuções de `./sync_car`	OK: `ok=100 fail=0`	—	03/11 18:46	`ok=100`
5	S1	Compilar com `-fsanitize=thread` (comando na Seção 3)	OK (warning só do runner)	—	03/11 (17:51–18:54)	`fsanitize=thread`
6	S1	`./sync_car_tsan`	Falha aparente, não era bug: `FATAL: ThreadSanitizer: unexpected memory mapping 0x710c1a672000-0x710c1ab00000`	Hipótese: a randomização de endereços (ASLR) do kernel 7.0.0-28 gera um mapeamento que o TSan do gcc 13.3 não aceita; independe de `sync_car.c`	03/11 (17:51–18:54)	`unexpected memory mapping`
7	S1	`setarch "$(uname -m)" -R ./sync_car_tsan` (ASLR desligado só nesse processo)	OK, com ressalva: `Looks good!` e 4 avisos (2 data races, 2 *thread leaks*), todos em `sync_car-runner.c`; nenhum em `sync_car.c`	Variáveis `volatile` compartilhadas sem proteção e threads sem `pthread_join`, ambos no runner (arquivo do professor, não alterado)	03/11 18:51	`SUMMARY: ThreadSanitizer`
8	S1	50 execuções de `sync_car_tsan` com `setarch -R`; critério: terminar com `Looks good` e nenhum aviso citando `sync_car.c`	OK: `problemas=0 de 50`	—	03/11 18:53	`problemas=0 de 50`
9	S1	`valgrind --tool=helgrind ./sync_car` (runner original)	Falha (diagnóstico inicial errado, revisto no item 10): `Failed to complete after 2 seconds` e `Exiting thread still holds 1 lock` em `station_load_car`/`pthread_cond_broadcast`	Primeira hipótese: helgrind deixa tudo lento e o `alarm` de 2 s do runner estourou; o "lock ao sair" seria só o `exit(1)` do runner no meio da função	03/11 (18:55–19:06)	`Failed to complete after 2 seconds`

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
10	S1	Mesma coisa numa cópia do runner em `/tmp/hg`, com `alarm` de 120 s (`sed -i`); runner original intacto	Falha → corrigida no item 11: `Failed to complete after 120 seconds` no primeiro carro (12 assentos), com a thread do carro parada em `sync_car.c:44` (`pthread_cond_broadcast`)	Não era só lentidão. Hipótese: o valgrind executa uma thread por vez, e a espera ocupada da `main` do runner impede a thread do carro de rodar	03/11 (18:55–19:06)	`Failed to complete after 120 seconds`
11	S1	`valgrind --tool=helgrind --fair-sched=yes ./sync_car_hg`, na cópia	OK: `Looks good!`; `ERROR SUMMARY: 65 errors from 4 contexts` (a conferência dos avisos está no item 12)	—	03/11 19:06	`65 errors from 4 contexts`
12	S1	Mesma execução com `2> /tmp/hg/err.txt` e `grep -c "sync_car\.c|station_" /tmp/hg/err.txt`	OK: `Looks good!` e `grep -c` igual a `0`: nenhum aviso envolve `sync_car.c` ou `station_*`	—	03/11 19:06	trecho final do log
13	S1	`valgrind --tool=helgrind ./sync_car` de novo, runner original, dentro do `script`	Falha reproduzida: `Failed to complete after 2 seconds` no primeiro carro (12 assentos); `Exiting thread still holds 1 lock`; `1 errors from 1 contexts`	Mesma hipótese do item 10; reproduziu	03/11 19:15	`==45155==`
14	S1	Helgrind na cópia com `alarm` de 120 s, sem `--fair-sched`	Falha reproduzida: `Failed to complete after 120 seconds`; `52 errors from 5 contexts`, que são data races em `alarm_timeout` e `alarm_error_str`	Variáveis do runner, acusadas porque o `alarm_handler` roda na thread parada em `station_wait_for_car` (`sync_car.c:26`); esse frame é só onde a thread bloqueou, não uma race em `sync_car.c`	03/11 19:19	`==45512==`
15	S1	`taskset -c 0 ./sync_car` (uma CPU, sem valgrind)	OK: `Looks good!`, uma execução	—	03/11 (19:20–19:21)	`taskset -c 0`
16	S1	Teste próprio de casos limite: `make teste_limite && ./teste_limite`, 8 casos	OK: 8 `PASS` (`count=0` e `count=10` sem passageiros; `count=0` com 5 esperando; passageiros > assentos; assentos > passageiros; assentos = passageiros; 1 passageiro e 1 assento; `count=-1` com 4 esperando)	—	03/11 19:33	`casos passaram`
17	S1	100 execuções de `./teste_limite`	OK: `ok=100 fail=0`	—	03/11 19:34	`teste_limite > /dev/null`

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
18	S1	`top -H -b -n 3 -d 2 -p $(pgrep teste_busy)` para checar *busy-wait*, duas vezes, sem `teste_busy` existir	Falha → corrigida: `top: option requires an argument -- 'p'`	`pgrep` devolveu vazio e deixou o `-p` sem argumento: o alvo precisa estar rodando antes do `top`	04/11 (09:11–09:14)	`option requires an argument`
19	S1	`teste_busy A`: 20 passageiros em `station_wait_for_car`, nenhum carro chega; `top -H -b -n 3 -d 2` 2 s depois	OK: 21 threads (20 passageiros e a `main`), `0 running, 21 sleeping`; todas em `S` e 0,0 % de CPU nas 3 amostras	—	04/11 09:13	`cenario A`
20	S1	`teste_busy B`: carro com 5 assentos chega; os 5 passageiros que reservam assento não chamam `station_on_board` (carro bloqueado em `all_boarded`) e os outros 15 voltam a dormir	OK: 22 threads, `0 running, 22 sleeping`; todas em `S` e 0,0 % de CPU nas 3 amostras	—	04/11 09:13	`cenario B`

6 Evidências

O `evidencias.log` foi gerado com `script -a` e acompanha o `.tar`; além dele há o histórico do Git e os prints de `prints/`.

O log foi limpo de códigos de escape e de linhas repetidas, sem alterar comandos nem saídas, com uma exceção: a linha `./teste_busy B &` do item 20 foi reconstruída, pois o eco se perdeu na edição com as setas. A saída do programa (`cenario B: pid=71492`) e o `Done ./teste_busy B` a sustentam.

Erro principal da semana: o travamento do `helgrind` (itens 9, 10, 13 e 14, Figura 1). O runner estourava o `alarm` mesmo com 120 s, o que descartou a hipótese de lentidão. Com `--fair-sched=yes` ele terminou em `Looks good!` (itens 11 e 12), o que sugere que o valgrind, ao serializar as threads, deixava a espera ocupada da `main` do runner ocupar a CPU e impedir o carro de rodar. Não verifiquei o escalonador do valgrind diretamente. O erro de código real foi o `cond_wait` sem o prefixo `pthread_` (item 1).

Validação final: runner em `Looks good!` e 100 de 100 execuções (itens 3 e 4, Figura 2); casos limite 8 de 8 (itens 16 e 17, Figura 3).

6.1 Prints

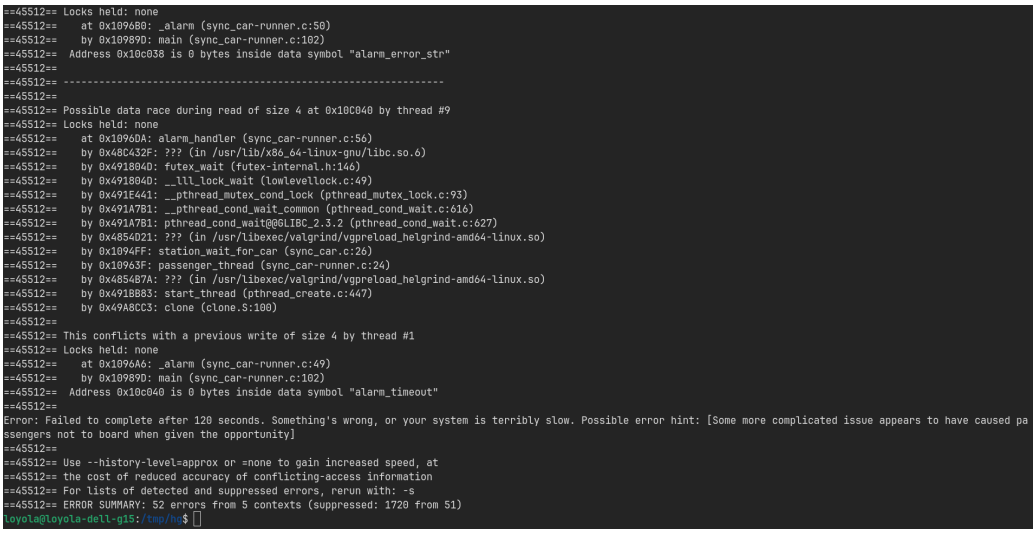


Figura 1: Erro principal da semana: `helgrind` na cópia do runner com `alarm` em 120 s e sem `--fair-sched`, que termina com `Failed to complete after 120 seconds` e `52 errors from 5 contexts` (item 14, 03/11 19:19). As races são em `alarm_timeout` e `alarm_error_str`, variáveis do runner. A espera maior não resolveu, o que derrubou a hipótese de lentidão do item 9; a falha com o `alarm` original de 2 s é o item 13.

```
loyola@loyola-dell-g15:~/Documents/blgv$ ./sync_car
car entering station with 15 free seats
car departed station with 15 new passenger(s) (expected 15)
car entering station with 3 free seats
car departed station with 3 new passenger(s) (expected 3)
car entering station with 41 free seats
car departed station with 41 new passenger(s) (expected 41)
car entering station with 35 free seats
car departed station with 35 new passenger(s) (expected 35)
car entering station with 20 free seats
car departed station with 6 new passenger(s) (expected 6)
Looks good!
loyola@loyola-dell-g15:~/Documents/blgv$ ok=0; fail=0
for i in $(seq 1 100); do
    if ./sync_car > /dev/null 2>&1; then
        ok=$((ok+1))
    else
        fail=$((fail+1)); echo "execucao $i FALHOU"
    fi
done
echo "ok=$ok fail=$fail"

ok=100 fail=0
```

Figura 2: Validação final: cada carro parte com o número esperado de passageiros, com `Looks good!` e `ok=100 fail=0` (itens 3 e 4, 03/11 18:46).

```
loyola@loyola-dell-g15:~/Documents/blgv$ make teste_limite && ./teste_limite
gcc -Wall -Wextra -pthread -o teste_limite teste_limite.c sync_car.c
PASS: count=0, sem passageiros
PASS: sem passageiros, count=10
PASS: count=0 com 5 passageiros esperando
PASS: passageiros(10) > assentos(3)
PASS: assentos(10) > passageiros(3)
PASS: assentos == passageiros (5)
PASS: 1 passageiro, 1 assento
PASS: count=-1 (invalido) com 4 esperando
```

Figura 3: Casos limite: 8 `PASS`, incluindo o caso inválido `count=-1 (invalido)` com 4 esperando (item 16, 03/11 19:33).

Limitações:

(1) o TSan rodou com ASLR desligado (`setarch -R`);

(2) no helgrind usei uma cópia do runner (só o `alarm` mudou) e `--fair-sched=yes`, porque o original dá *timeout*;

(3) a checagem de *busy-wait* (itens 19 e 20) cobre 3 amostras em 2 cenários com as threads mantidas bloqueadas de propósito, sem o fluxo normal de embarque.

Dificuldades: todos os requisitos do enunciado foram implementados; as dificuldades foram de ferramenta e não de projeto: o ASLR do kernel impedia o TSan de iniciar (item 6) e o helgrind travava o runner (itens 9, 10, 13 e 14).

7 Análise: ausência de *lost wakeup* e de *busy-wait*

Passageiros. `station_load_car` grava `car_present = 1` e `free_seats = count` com o mutex preso e só então faz o `broadcast`; os passageiros reavaliam o `while` com o mutex de volta. Quem chega com carro presente e assento livre nem dorme; quem chega com o carro cheio dorme em `car_arrived` e acorda no próximo `broadcast`.

Carro. Dorme em `all_boarded` enquanto vale `(free_seats > 0 && waiting > 0) || boarding > 0`. Invariante: se o carro dorme, ainda há um passageiro que vai chamar `station_on_board`, e o último a chamá-lo zera `boarding` e emite o sinal.

Sem espera ocupada. Toda espera é em `pthread_cond_wait`, e o `top` mostrou 0,0 % de CPU com as threads bloqueadas (itens 19 e 20). Este é um argumento de projeto, sustentado pelos itens 4, 8, 11, 12, 19 e 20, e não uma garantia para todas as execuções possíveis.

Em suma: a condição é mudada sob o mutex, o `broadcast` vem depois da mudança e o carro só dorme quando algum passageiro ainda vai chamar `station_on_board`.

8 Uso de IA

Onde usei IA: usei o Claude para revisar meu `sync_car.h` e meu `sync_car.c`, gerar o `Makefile` e o checklist, e orientar os comandos de teste. A revisão apontou erros de sintaxe em `station_init`, o `cond_wait` sem `pthread_` (item 1) e um `NULL` no lugar do mutex em `pthread_cond_wait`. O que escrevi: `sync_car.h` e as quatro funções de `sync_car.c`. O que a IA escreveu: o `Makefile`, o checklist e o programa `teste_busy.c` dos itens 19 e 20; o Claude Code também limpou os códigos de escape do `evidencias.log`.

Prompts principais: como funcionam `pthread_mutex_lock` e `pthread_mutex_unlock`; discussão das hipóteses para o `FATAL: ThreadSanitizer` (ASLR, item 6) e para o travamento do helgrind (escalonamento do valgrind, itens 10 e 13).

O que validei manualmente: compilei com `-Wall -Wextra -pthread`, executei todos os testes do diário (itens 2 a 20) e li de qual arquivo vinha cada aviso (só do runner, nunca de `sync_car.c`). As hipóteses discutidas com a IA foram testadas por execução (itens 7, 11 e 12) e não apenas aceitas.

9 Reflexão Final

A lição da semana foi desconfiar do meu diagnóstico do helgrind: tratei o travamento como lentidão (itens 9 e 10) e só a execução com `alarm` de 120 s mostrou que a hipótese estava errada, o que levou a `--fair-sched=yes` (itens 11 e 12). A decisão mais acertada foi reservar o assento dentro de `station_wait_for_car`, junto com o `while` em todo `pthread_cond_wait`, que tornou irrelevante a ordem em

que os passageiros acordam: cada um reavalia a condição com o mutex preso. O erro de código que quero evitar é o do item 1, o nome do pseudocódigo (`cond_wait`) no lugar de `pthread_cond_wait`: da próxima vez escrevo cada função a partir da lista de funções permitidas do enunciado e compilo com `-Wall -Wextra` a cada função.

10 Checklist Final de Entrega

Item	**Confirmado?**
Compilei do zero (`make clean && make`), só com o que está na Seção 3	Sim (item 2)
Rodei os testes e o `evidencias.log` foi gerado	Sim (itens 3 a 20)
Tenho os prints obrigatórios: erro principal (Figura 1) e validação final (Figura 2)	Sim
Testei casos limite e um caso inválido (`count=-1`)	Sim (itens 16 e 17)
Preenchi a seção de uso de IA, separando o que escrevi do que a IA escreveu	Sim
Revisei o relatório e removi frases genéricas ou vazias	Sim

11 Se eu tivesse mais 2 horas

Investiria em: (1) incluir `teste_limite` e `teste_busy` na entrega e criar um alvo `make test` que rode o loop de 100 execuções do runner e os casos limite, hoje feitos à mão e não reproduzíveis a partir do `.tar`; (2) estressar contra *lost wakeup* com 1000 passageiros e carros em sequência rápida, em 1000 execuções e com `taskset -c 0` para variar o escalonamento, aceitando só se nenhuma travar ou estourar o `alarm`, já que o argumento da Seção 7 não exclui esse bug; (3) repetir a checagem de *busy-wait* com um embarque completo, medindo o `TIME+` acumulado do processo em vez do `%CPU` instantâneo (limitação 3).